

TypeScript

O [TypeScript](#) não é uma linguagem de programação, mas sim uma ferramenta criada pela Microsoft e nada mais é do que superset do ECMAScript 6. Isso significa que praticamente qualquer código JavaScript é também um código TypeScript.

Ele adiciona tipagem estática ao JavaScript que, por padrão, é uma linguagem que possui tipagem dinâmica, ou seja, as variáveis e funções podem assumir tipos distintos durante o tempo de execução.

É utilizado somente em ambiente de desenvolvimento e é totalmente convertido para JavaScript no processo de build de produção, uma vez que o navegador interpreta apenas linguagem JS. Desta forma, o TypeScript nunca altera o comportamento do programa com base nos tipos que ele infere, não tem influência sobre como o programa funciona quando executado.

Necessário ter o Node.js instalado, uma vez que sem ele o compilador do TypeScript não funciona.

Com essa ferramenta temos a possibilidade de descobrir erros durante o desenvolvimento e incrementar a inteligência (*IntelliSense*) da IDE que estamos utilizando.

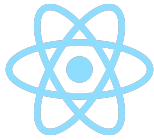
Comparando softwares escritos em JavaScript e TypeScript (mesmas funcionalidades), é possível afirmar que a versão em TS terá uma manutenção potencialmente mais simples, uma vez que as IDEs oferecem mais possibilidades de assistência de codificação para um código TS do que para JS.

Importante!! *TypeScript não é substituto do JavaScript, mas sim uma ferramenta construída baseada em JavaScript.*

Tipagem Estática vs. Tipagem Dinâmica

A tipagem dinâmica está ligada a habilidade da linguagem de programação em escolher o tipo de dado de acordo com o valor atribuído à variável em tempo de execução dinamicamente. Tipagem acontece em tempo de execução, erros são descobertos apenas executando a aplicação. Alguns exemplos de linguagens com tipagem dinâmica: JavaScript, Python, PHP, Ruby.

Já a tipagem estática concede uma maior segurança ao código, já que garante que o build não seja gerado caso ocorra algum erro de tipagem. Não permite alterar o tipo da variável depois de declarada. Geralmente a verificação é feita em tempo de compilação e quando tentamos atribuir um valor de um tipo diferente do que foi declarado na variável temos um erro. Alguns exemplos de linguagens com tipagem estática: C, C++, Java, Go e Rust.



Tipagem Estática vs. Tipagem Dinâmica

Tipagem fraca está ligada a características da linguagem de realizar conversões automaticamente entre diferentes tipos de dados.

```
var nome = "Elton Fonseca"; //string
var idade = 28; //number

console.log(nome + " " + idade); //Elton Fonseca 28
```

Nas linguagens fortemente tipadas, os tipos são muito importantes, portanto é necessário sempre informá-los, uma vez que esse tipo de linguagem não realiza conversões automaticamente.

```
var nome = "Elton Fonseca"; //str
var idade = 28; //int

console.Log(nome + " " + idade);
#TypeError: can only concatenate str (not "int") to str
```

De onde vem os tipos?

As bibliotecas com TypeScript incluem um arquivo de definição de tipos (extensão .d.ts) gerado automaticamente. Esse arquivo guarda a informação de tipagem de todas variáveis, funções, classes, que são exportadas pela biblioteca.

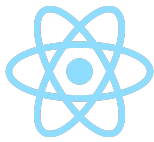
Na grande maioria das vezes, esses tipos são instalados separadamente da biblioteca principal.

```
yarn add react
yarn add @types/react -D
```

Preciso tipar todas as variáveis e funções?

No desenvolvimento do projeto, o TypeScript executa o código e encontra todas entradas e saídas possíveis.

Através da inferência de tipos, ele consegue determinar automaticamente o tipo de algumas variáveis, parâmetros e retornos das funções. Apenas é necessário definir tipagens quando o TypeScript não conseguir definir isso de forma automática.



Veja o exemplo abaixo:

```
const a = 5;
```

Neste caso, o TypeScript entende que a variável `a` é numérica e, se tentarmos alterar seu valor para uma string, receberemos um erro.

Uma ferramenta que pode auxiliar no uso do TypeScript é o *EsLint*, que irá “avisar” quando não for possível inferir a tipagem de forma automática.

The screenshot shows a code editor with a TypeScript file. On line 16, there is a function component definition: `const SidebarItem = ({ id }) => {`. The `id` prop is not typed. On line 17, the `return` statement is partially visible. Below the code editor, the 'TERMINAL' tab is active, displaying an error message: `ERROR in src/components/Sidebar/index.tsx:16:24` and `TS7031: Binding element 'id' implicitly has an 'any' type.`

Formas de tipagem

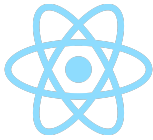
Simples:

- Variáveis

```
const x: number = 3;  
const vetor: number[] = [1, 2, 3];
```

- Função

```
function compare (x: number, y: number): string {  
  return x > y ? "X maior que Y" : "Y maior que X";  
}
```



- Interface

Para reaproveitarmos tipagens entre vários arquivos e funções da aplicação. Podem herdar outras interfaces.

```
interface Pessoa {  
  nome: string  
  idade: number  
}  
  
function bomDia (pessoa: Pessoa): string {  
  return `Bom dia ${pessoa.nome}`;  
}  
  
function maiorDeIdade (pessoa: Pessoa): boolean {  
  return pessoa.idade >= 18;  
}
```

- Types

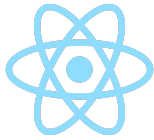
Quando uma variável pode assumir formatos distintos mesmo pertencendo a uma mesma entidade. No exemplo a seguir, o parâmetro da função pode assumir três formatos diferentes baseado no tipo de figura geométrica.

```
type Polygon =  
  { type: "square", x: number } ||  
  { type: "circle", radius: number } ||  
  { type: "rectangle", x: number, y: number};  
  
export function area (polygon: Polygon) : number {}
```

Quais são os tipos?

Os dados podem ser dos seguintes tipos:

- undefined
- null
- boolean
- number
- string
- symbols
- objects
- any
- unknow
- array
- void



Any: equivalente a uma variável escrita em JavaScript. O tipo any pode receber qualquer valor.

```
let a: any = "anything";
```

Boolean: recebe os valores de true ou false.

```
let a: boolean = true;
```

Number: este tipo é um pouco diferente do que normalmente é visto em outras linguagens. O TypeScript não dispõe de números inteiros, sem sinal ou algo do tipo. Todos os números são definidos como números reais e podem ser representados, inclusive, por binários, hexadecimais, etc.

```
let a: number = 10  
a = 10.2
```

String: esse tipo possui a variação Template Strings, onde é possível quebrar linhas e inserir variáveis muito mais facilmente do que no JavaScript. Para isso, basta abrir a string com o caractere acento grave (`).

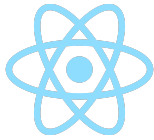
```
let message: string = "Hello World";  
let a: string = `  
  <div>  
    <p>${message}</p>  
  </div>  
`;  
`;
```

Array: há duas formas de se utilizar um array, sendo que ambas representam a mesma coisa. A primeira se trata de uma sintaxe sugar da segunda, que utiliza o recurso chamado Generics, muito comum em outras linguagens.

```
let a: any[];  
let b: Array<any>;
```

Também é possível termos um array que recebe mais de um tipo, nesse caso ao declarar é necessário informar quais tipos de dados esse array estará apto a receber.

```
let carros: (string || number)[] = ["Gol", 2014, "Fiesta", 2018];
```



Tuple: semelhante a um array, porém com tamanhos e valores de tipos bem definidos. São bastante úteis, flexíveis, porém pouco explorados.

```
let a: [string, any][] = [  
  ["url", "http://localhost"],  
  ["port", 8080]  
]
```

Funções:

- *Void*: ao contrário do *any* (qualquer tipo de dado), é possível especificar um tipo *void* (nenhum tipo de dado), que é muito utilizado em funções para informar que ela não retorna nada.

```
function message(): void {  
  alert("hey ho let's go")  
}
```

- Retorno de dados: é possível também informar um tipo para o retorno da função.

```
function isChecked(): boolean {  
  return true;  
}
```